

ບົດທີ 8: ພື້ນຖານຂອງ DevOps

Fundamentals of DevOps



ການທຳລາຍກຳແພງ

Dev ແລະ Ops



ວັດທະນະທຳ

CALMS



ການເຊື່ອມໂຍງ

CI/CD



ການຫຸ້ມຫໍ່

Docker

ເນື້ອໃນຫຍໍ້

ພື້ນຖານຂອງ DevOps - ຫົວຂໍ້ທີ່ຈະສຶກສາ



01

ກຳແພງແຫ່ງຄວາມສັບສົນ: ຄວາມຂັດແຍ່ງ
ລະຫວ່າງ Dev ແລະ Ops



02

DevOps ແມ່ນຫຍັງ: ວັດທະນະທຳ
CALMS ແລະ ແນວທາງປະຕິບັດ



03

Continuous Integration (CI): ແນວຄິດ,
ຂະບວນການ, ແລະ ເຄື່ອງມື



04

Continuous Delivery/Deployment (CD):
ແນວຄິດ ແລະ ຂະບວນການ



05

Docker ແລະ Containerization: ແກ້ໄຂ
ບັນຫາ "It works on my machine!"



06

Cloud Computing: ຮູບແບບການບໍລິການ
ແລະ ຄວາມກ່ຽວຂ້ອງກັບ DevOps

ກຳແພງແຫ່ງຄວາມສັບສົນ

ຄວາມຂັດແຍ່ງລະຫວ່າງທີມ Dev ແລະ Ops

<> ທີມພັດທະນາ (Dev)

ເປົ້າໝາຍ:

ສ້າງ Feature ໃໝ່ໃຫ້ໄວທີ່ສຸດ

"It works on my machine!
ຢູ່ເຄື່ອງຂ້ອຍມັນເຮັດວຽກໄດ້!"

ກຳແພງແຫ່ງຄວາມສັບສົນ

"ເຮົາສ້າງ Feature ໃໝ່ສໍາເລັດແລ້ວ,
ໂຍນຂ້າມກຳແພງໄປໃຫ້ Ops ເອົາຂຶ້ນ
Server ເອງ."

⚙️ ທີມປະຕິບັດການ (Ops)

ເປົ້າໝາຍ:

ຮັກສາສະຖຽນລະພາບ 24/7

"Your code broke production!
ໂຄດຂອງເຈົ້າເຮັດໃຫ້ Server ພັງ!"



ຂະບວນການສິ່ງມອບຊຸກຊ້າ



ເກີດຄວາມຜິດພາດບໍ່ຄາດຄິດ



ເຕັມໄປດ້ວຍຄວາມຂັດແຍ່ງ

DevOps: ການທຳລາຍກຳແພງ

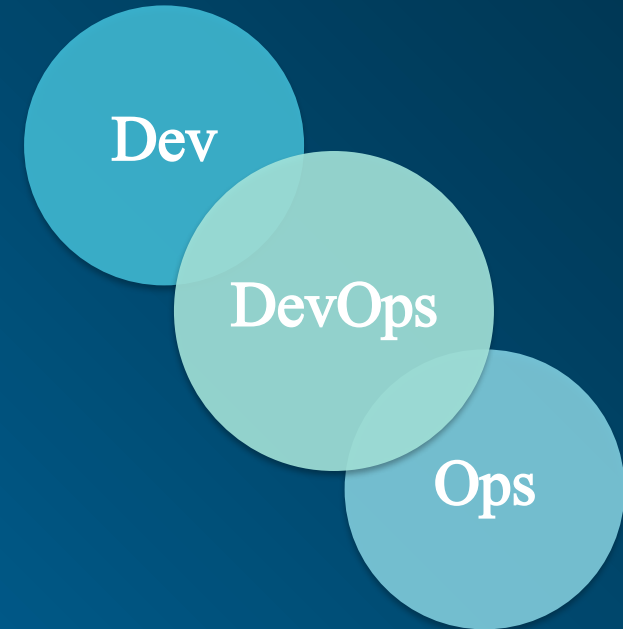
ວັດທະນະທຳ ແລະ ແນວທາງປະຕິບັດ

DevOps ບໍ່ແມ່ນ...

- ⊗ ...ຕຳແໜ່ງ ຫຼື ທີ່ມ ໃນອົງກອນ
- ⊗ ...ເຄື່ອງມື ຫຼື ເຕັກນິກ ທີ່ສາມາດຊື້ຫຼືຕິດຕັ້ງໄດ້

DevOps ແມ່ນ...

- ⚙️ ວັດທະນະທຳ ທີ່ສົ່ງເສີມການຮ່ວມມື ແລະ ການຮັບຜິດຊອບຮ່ວມກັນ
- ⚙️ ຊຸດຂອງແນວທາງປະຕິບັດ ທີ່ມຸ່ງເນັ້ນການສົ່ງມອບຊອບແວທີ່ມີຄຸນຄ່າໃຫ້ຜູ້ໃຊ້ຢ່າງຕໍ່ເນື່ອງ
- ☑️ ຊ່ວຍໃຫ້ທັງສອງຝ່າຍສາມາດຮ່ວມມືກັນ ເພື່ອສົ່ງມອບຊອບແວໄວ, ບໍ່ຫຼຸດພັນ, ແລະ ໜ້າເຊື່ອຖື



"DevOps ຄືການປ່ຽນແປງວັດທະນະທຳຂອງອົງກອນ ທີ່ຕ້ອງເລີ່ມຈາກການທຳລາຍກຳແພງ ແລະ ສົ່ງເສີມການຮ່ວມມື"

ວັດທະນະທຳ DevOps: CALMS

ແຖນຫຼັກຂອງ DevOps ຄືການປ່ຽນ Mindset



C

Culture (ວັດທະນະທຳ)

ສົ່ງເສີມການຮ່ວມມື ແລະ ການ
ຮັບຜິດຊອບຮ່ວມກັນ

ຮ່ວມມື ຮັບຜິດຊອບ ບໍ່ມີກຳແພງ



A

Automation (ອັດຕະໂນມັດ)

ເຮັດທຸກຢ່າງທີ່ເຮັດຊ້ຳໃຫ້ເປັນອັດຕະ
ໂນມັດ ເພື່ອຫຼຸດຜ່ອນຄວາມຜິດພາດ

CI/CD ການທົດສອບການຈັດການ



L

Lean (ຄວາມກະທັດຮັດ)

ສຸມໃສ່ການສ້າງຄຸນຄ່າ ແລະ ກຳຈັດ
ສິ່ງທີ່ສຸນເສຍໃນຂະບວນການ

ຫຼຸດຂີ້ເຫຼີ ປັບປຸງຕະຫຼອດສ້າງຄຸນຄ່າ



M

Measurement (ການວັດແທກ)

ເກັບຂໍ້ມູນ ແລະ ວັດແທກທຸກຢ່າງ ເພື່ອນຳໄປໃຊ້ໃນການ
ຕັດສິນໃຈ

ຕົວຊີ້ວັດການຕິດຕາມການປັບປຸງ



S

Sharing (ການແບ່ງປັນ)

ສົ່ງເສີມການແບ່ງປັນຄວາມຮູ້, ເຄື່ອງມື, ແລະ ຄວາມ
ຮັບຜິດຊອບ

ຄວາມຮູ້ເຄື່ອງມື ປະສົບການ

"CALMS ແມ່ນພື້ນຖານຂອງວັດທະນະທຳ DevOps ທີ່ຊ່ວຍໃຫ້ອົງກອນສາມາດ ສົ່ງມອບຊອບແວທີ່ມີຄຸນຄ່າໃຫ້ລູກຄ້າໄດ້ຢ່າງຕໍ່ເນື່ອງ"

ແນວທາງປະຕິບັດຫຼັກຂອງ DevOps

ການນຳໃຊ້ເຕັກນິກ ແລະ ແນວທາງປະຕິບັດເພື່ອສົ່ງມອບຄຸນຄ່າໃຫ້ລູກຄ້າ



CI/CD

Continuous Integration & Delivery/
Deployment

ການລວມໂຄດ ແລະ ການສົ່ງມອບຊອບແວແບບຕໍ່ເນື່ອງ

- ✓ ລວມໂຄດທຸກມື້
- ✓ ທົດສອບອັດຕະໂນມັດ
- ✓ ປ່ອຍເວີຊັນນ້ອຍໆ



IaC

Infrastructure as Code

ການຈັດການ Server ແລະ ສະພາບແວດລ້ອມຜ່ານການຂຽນໂຄດ

- ✓ ຈັດການອັດຕະໂນມັດ
- ✓ ຄວາມສົມດຸນ
- ✓ Version Control



Monitoring

ການຕິດຕາມ ແລະ ການເກັບກຳຂໍ້ມູນ

ການຕິດຕາມສະຖານະຂອງລະບົບ ແລະ ການເກັບກຳຂໍ້ມູນການເຮັດວຽກ

- ✓ ການຕິດຕາມສະຖິຕິ
- ✓ ການແຈ້ງເຕືອນ
- ✓ ການວິເຄາະຫາສາເຫດ



Containerization

ການຫຸ້ມຫໍ່ແອັບພລິເຄຊັນ

ການຫຸ້ມຫໍ່ແອັບພລິເຄຊັນ ແລະ dependencies ທັງໝົດໄວ້ນຳກັນ

- ✓ ຄວາມສົມດຸນຂ້າມສະພາບແວດລ້ອມ
- ✓ ການຂະຫຍາຍຕົວງ່າຍ
- ✓ ຫຼຸດຄວາມສັບສົນ

"ແນວທາງປະຕິບັດເຫຼົ່ານີ້ຊ່ວຍໃຫ້ອົງກອນສາມາດ ສົ່ງມອບຊອບແວໄວ, ປອດໄພ, ແລະ ມີຄຸນນະພາບສູງ"

Continuous Integration (CI)

ການເຊື່ອມໂຍງແບບຕໍ່ເນື່ອງ

🔧 ແນວຄິດຫຼັກ

CI ແມ່ນແນວທາງປະຕິບັດທີ່ນັກພັດທະນາທຸກຄົນຈະ **ລວມໂຄດ** ຂອງຕົນເອງເຂົ້າກັບໂຄດຫຼັກ **ຢູ່ສະເໝີ** — ໂດຍທົ່ວໄປຄືຫຼາຍໆຄັ້ງຕໍ່ມື້

- 1 Developer push code ໃໝ່ຂຶ້ນໄປເທິງ Git Repository
- 2 CI Server ເຮັດການ Build ໂດຍອັດຕະໂນມັດ
- 3 ລັ່ງ Unit Tests ແລະ Integration Tests ທັງໝົດ
- 4 ຖ້າຜ່ານ: ແຈ້ງ "Build is green" ✓
- 5 ຖ້າລົ້ມ: ແຈ້ງ "The build is broken!" ✗

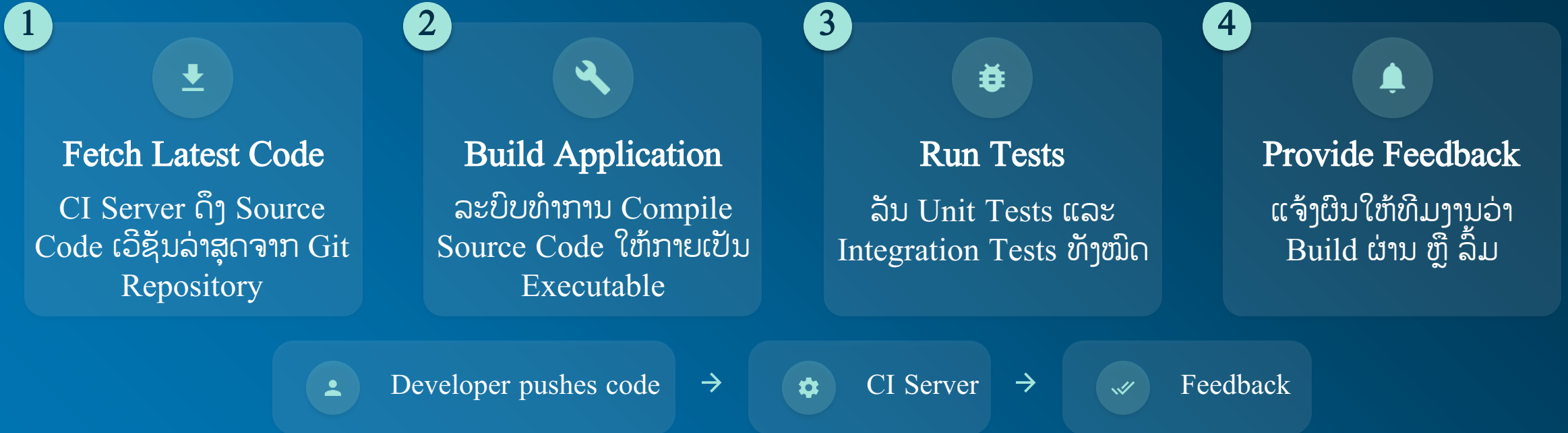
🌟 ປະໂຫຍດຂອງ CI

- 🛡️ ຫຼຸດຜ່ອນຄວາມສ່ຽງ
ຊ່ວຍໃຫ້ກວດພົບ Bug ແລະ ບັນຫາການເຊື່ອມໂຍງໄດ້ໄວທີ່ສຸດ
- 📄 ເພີ່ມຄວາມໄວ
ນັກພັດທະນາບໍ່ຕ້ອງເສຍເວລາໃນການແກ້ໄຂ Integration Hell
- ⚙️ ສ້າງຄວາມໝັ້ນໃຈ
ເມື່ອ Build ເປັນສີຂຽວ, ທຸກຄົນໝັ້ນໃຈວ່າໂຄດຫຼັກຢູ່ໃນສະພາບທີ່ດີ
- 🏠 ພື້ນຖານຂອງ CD
ບໍ່ສາມາດສົ່ງມອບຊອບແວແບບຕໍ່ເນື່ອງໄດ້ຖ້າບໍ່ມີ CI

"CI ຄືການປ່ຽນຈາກການ 'ລວມໂຄດເປັນຄັ້ງຄາວ ແລະ ເຈັບປວດ' ໄປສູ່ການ 'ລວມໂຄດຢູ່ສະເໝີ ແລະ ເປັນອັດຕະໂນມັດ'"

CI Pipeline: ຂັ້ນຕອນ ແລະ ຂະບວນການ

ຂະບວນການທີ່ເຮັດວຽກອັດຕະໂນມັດທຸກຄັ້ງທີ່ມີການ push code



! ຈຸດສຳຄັນທີ່ຕ້ອງຈື່

- 🕒 ເມື່ອ Build ລົ້ມ, ການແກ້ໄຂ Build ທີ່ລົ້ມນັ້ນ ຈະກາຍເປັນວຽກທີ່ມີຄວາມສຳຄັນສູງສຸດ
- ✅ ເມື່ອ Build ຜ່ານ, ລະບົບຈະແຈ້ງ "Build is green" ✓
- 🔴 ຖ້າ Build ລົ້ມ, Pipeline ຈະຢຸດ ແລະ ແຈ້ງເຕືອນທີມງານທັນທີ
- 👥 ບໍ່ມີໃຜຄວນຈະເຮັດວຽກອື່ນຕໍ່ ຈົນກວ່າ Build ຈະກັບມາເປັນສີຂຽວ

CI Tools: ເຄື່ອງມືທີ່ນິຍົມໃຊ້

ເຄື່ອງມືທີ່ຊ່ວຍໃຫ້ການປະຕິບັດ CI ເປັນໄປໄດ້ຢ່າງງ່າຍດາຍ



Jenkins

- ✓ CI Server ແບບ Open Source ເກົ່າແກ່
- ✓ ມີພະລັງທີ່ສຸດ
- ✓ ມີ Plugins ໃຫ້ໃຊ້ງານຢ່າງຫຼວງຫຼາຍ

ມີປະສິບການຫຼາຍທີ່ສຸດ



GitHub Actions

- ✓ ສ້າງມາພ້ອມກັບ GitHub
- ✓ ງ່າຍຕໍ່ການເລີ່ມຕົ້ນ
- ✓ ເຮັດວຽກຮ່ວມກັນກັບ GitHub ໄດ້ງ່າຍ

ເຫມາະສົມກັບໂຄງການໃຫຍ່



GitLab CI/CD

- ✓ ສ້າງມາພ້ອມກັບ GitLab
- ✓ ມີຄວາມສາມາດສູງ
- ✓ ໄດ້ຮັບຄວາມນິຍົມຫຼາຍ

ຄວາມສາມາດສູງ



CircleCI

- ✓ ບໍລິການ CI/CD ບິນ Cloud
- ✓ ມີແຜນຟຣີ ແລະ ສະຫນັບສະຫນູນ Docker
- ✓ ງ່າຍຕໍ່ການຕັ້ງຄ່າ

ຄວາມສະດວກ



Travis CI

- ✓ ບໍລິການ CI/CD ເທິງ Cloud
- ✓ ສະຫນັບສະຫນູນຫຼາຍພາສາ
- ✓ ມີແຜນຟຣີ ສໍາລັບໂຄງການໂອເຟັນຊອດ

ເຫມາະສົມໂຄງການໂອເຟັນຊອດ



ເລືອກເຄື່ອງມື?

- ✓ ຂະໜາດໂຄງການ
- ✓ ທັກສະຂອງທີມ
- ✓ ງົບປະມານ
- ✓ ຄວາມຕ້ອງການຂອງລະບົບ

ຕາມຄວາມເໝາະສົມ

"ເຄື່ອງມື CI ເຫຼົ່ານີ້ຊ່ວຍໃຫ້ທີມສາມາດ ປະຕິບັດ CI ໄດ້ຢ່າງງ່າຍດາຍ ໂດຍບໍ່ຕ້ອງສ້າງລະບົບຈາກຕົ້ນ"

Continuous Delivery vs. Continuous Deployment

ຄວາມແຕກຕ່າງທີ່ສໍາຄັນຢູ່ຂັ້ນຕອນສຸດທ້າຍ



Continuous Delivery

ການສົ່ງມອບແບບຕໍ່ເນື່ອງ

ທຸກໆການປ່ຽນແປງທີ່ຜ່ານຂະບວນການທົດສອບຈະຖືກ Deploy ໄປຍັງສະພາບແວດລ້ອມທີ່ຄ້າຍຄື Production ໂດຍອັດຕະໂນມັດ

- 🔧 ການ Deploy ຂຶ້ນ Production ຍັງຄົງເປັນ **ການຕັດສິນໃຈທາງທຸລະກິດ** ທີ່ຕ້ອງມີຄົນກົດປຸ່ມ
- ⚙️ ເປົ້າໝາຍ: ຮັບປະກັນວ່າມີ **Candidate Release** ທີ່ພ້ອມຈະປ່ອຍ
- 🕒 ຕົວຢ່າງ: ແອັບ Mobile Banking ອາດຈະລໍຖ້າຈົນເຖິງຊ່ວງເວລາທີ່ຜູ້ໃຊ້ນ້ອຍກ່ອນຈະກົດປຸ່ມ Deploy



Continuous Deployment

ການນໍາໃຊ້ແບບຕໍ່ເນື່ອງ

ທຸກໆການປ່ຽນແປງທີ່ຜ່ານຂະບວນການທົດສອບຈະຖືກ Deploy ຂຶ້ນ Production ໃຫ້ຜູ້ໃຊ້ໄດ້ໃຊ້ໂດຍອັດຕະໂນມັດ

- ✨ **ບໍ່ມີການກົດປຸ່ມໂດຍຄົນ** ຖ້າ Pipeline ເປັນສີຂຽວ, ມັນຈະຂຶ້ນ Production ທັນທີ
- 📄 ເປົ້າໝາຍ: ຫຼຸດໄລຍະເວລາຈາກແນວຄວາມຄິດໄປສູ່ Feature ທີ່ຢູ່ໃນມືລຸກຄ້າ
- ⚡ ຕົວຢ່າງ: Developer ແກ້ໄຂ Bug ການສະແດງຜິດເລັກໆ, 15 ນາທີຕໍ່ມາ ໂຄດໃໝ່ກໍຖືກປ່ອຍໃຫ້ຜູ້ໃຊ້ແລ້ວ



Code



Build



Test



Deploy to Staging



ຈຸດແຕກຕ່າງ



Manual
Button Push



Automatic



Production

"CD ຄືເປົ້າໝາຍສູງສຸດຂອງທີມ Agile ແລະ DevOps, ແຕ່ຕ້ອງການການລົງທຶນອັດຕະໂນມັດໃນການທົດສອບຢ່າງຫຼວງຫຼາຍ"

Deployment Pipeline: ເສັ້ນທາງສູ່ Production

ຂະບວນການທັງໝົດ ຕັ້ງແຕ່ CI ໄປຈົນເຖິງ CD

-  **Commit Stage (CI)**
Developer push code → CI Server Build → Run Tests → Create Package
 - ✓ Unit Tests
 - ✓ Integration Tests
 - ✓ Artifact Creation
-  **Acceptance Testing Stage**
Deploy ໄປ Staging Environment → Run System Tests → End-to-End Tests
 - ✓ Automated Tests
 - ✓ Production-like Environment
-  **Capacity/Performance Testing Stage**
Run Performance Tests → ໃຫ້ແນ່ໃຈວ່າການປ່ຽນແປງໃໝ່ບໍ່ໄດ້ເຮັດໃຫ້ລະບົບຊ້າລົງ
 - ✓ Load Testing
 - ✓ Stress Testing
-  **Release/Deployment Stage**
Continuous Delivery: ລໍຖ້າການກວດກາຈາກຄົນ → Deploy to Production
Continuous Deployment: Deploy ອັດຕະໂນມັດ → ບໍ່ຕ້ອງລໍຖ້າ
 - ⚠ Manual Approval
 - ⚡ Automatic Deployment

"Deployment Pipeline ຊ່ວຍໃຫ້ທີມສາມາດ ສົ່ງມອບຊອບແວໄດ້ຢ່າງຕໍ່ເນື່ອງ ພ້ອມທັງການທົດສອບທີ່ຄົບຖ້ວນ"

ບັນຫາ "It Works on My Machine!"

ສາເຫດທີ່ໂຄດເຮັດວຽກໄດ້ໃນສະພາບແວດລ້ອມໜຶ່ງ ແຕ່ໄປລົ້ມໃນອີກສະພາບແວດລ້ອມໜຶ່ງ

Developer

Production



"It works **on my machine!**"

? ສາເຫດທີ່ເຮັດໃຫ້ເກີດບັນຫາ



OS ຕ່າງກັນ: Developer ໃຊ້ Windows 11, ແຕ່ Server ເປັນ Linux



Runtime ຕ່າງກັນ: Developer ໃຊ້ Node.js v18, Server ມີພຽງ v14



Dependencies ບໍ່ຄົບຖ້ວນ: ໂຄດຕ້ອງການ Library A, B, C, ແຕ່ Server ມີພຽງ A ແລະ B



ການຕັ້ງຄ່າບໍ່ຄືກັນ: Configuration ທີ່ແຕກຕ່າງກັນ ລະຫວ່າງສະພາບແວດລ້ອມ

"ບັນຫາເຫຼົ່ານີ້ເຮັດໃຫ້ຂະບວນການ Deploy ຊັກຊ້າ, ຄາດເດົາບໍ່ໄດ້, ແລະ ເຕັມໄປດ້ວຍຄວາມຂັດແຍ່ງ"

Docker: ການແກ້ໄຂບັນຫາ "It Works on My Machine"

ຫຼັກທຳທຸກຢ່າງໄວ້ນຳກັນ

Docker ຄືການແກ້ໄຂບັນຫາ

Docker ແມ່ນແຟລດຟອມ Open Source ທີ່ຊ່ວຍໃຫ້ເຮົາສາມາດ **ຫຼັກທຳ** ແອັບພລິເຄຊັນຂອງເຮົາ ພ້ອມກັບທຸກສິ່ງທີ່ມັນຕ້ອງການ ເພື່ອໃຫ້ສາມາດເຮັດວຽກໄດ້

- ✓ **ຫຼັກທຳ** ແອັບພລິເຄຊັນ ພ້ອມກັບ Libraries, System tools, Runtime, ແລະ ການຕັ້ງຄ່າ
- ✓ ເຮັດວຽກໄດ້ **ຄືກັນທຸກບ່ອນ** ບໍ່ວ່າຈະເອົາໄປລັນຢູ່ໃສ
- ✓ ແກ້ໄຂບັນຫາ **"It works on my machine!"** ໂດຍສິ້ນເຊີງ

ການປຽບທຽບ: ຕູ້ຄອນເທນເນອ໌

ກ່ອນ Docker



ການຂົນສົ່ງສິນຄ້າຕ່າງໆຊະນິດ
ໄປນຳກັນເປັນເລື່ອງທີ່ວຸ້ນວາຍ
ແລະ ມີຄວາມສ່ຽງສູງ

ຫຼັງ Docker



ເອົາສິນຄ້າຫຍັງກໍໄດ້
ໃສ່ໃນຕູ້ຄອນເທນເນອ໌
ຈັດການໄດ້ແບບດຽວກັນຫມົດ

✦ Docker Container ກໍຄືກັນກັບຕູ້ຄອນເທນເນອ໌, ຊ່ວຍຫຼັກທຳແອັບພລິເຄຊັນ ແລະ Dependencies ທັງໝົດໄວ້ນຳກັນ

"Docker ແລະ Containerization ໄດ້ປ່ຽນແປງວິທີການທີ່ເຮົາພັດທະນາ ແລະ Deploy ຊອບແວຢ່າງສິ້ນເຊີງ"

Docker Image vs. Docker Container

ຄວາມແຕກຕ່າງທີ່ສຳຄັນ



Docker Image

ແບບແປນ / Template

ໄຟລ໌ທີ່ບັນຈຸຄຳສັ່ງທັງໝົດທີ່ຈຳເປັນໃນການສ້າງ Container, ເປັນແບບ **Read-only**

- ✓ ເປັນ **Template** ທີ່ບໍ່ສາມາດປ່ຽນແປງໄດ້
- ✓ ມີລັກສະນະ **Layered** ເພື່ອປະຢັດເນື້ອທີ່
- ✓ ສາມາດແບ່ງປັນ ແລະ ສົ່ງຜ່ານເຄືອຂ່າຍໄດ້



Docker Container

Instance ທີ່ກຳລັງເຮັດວຽກ

Instance ທີ່ກຳລັງເຮັດວຽກຂອງ Image, ເປັນແບບ **Writable** ແລະ ແຍກກັນ

- ✓ ເປັນ **Instance** ທີ່ສາມາດສ້າງຫຼາຍອັນຈາກ Image ດຽວກັນໄດ້
- ✓ ມີສະຖານະ **Running, Stopped**, ຫຼື **Paused**
- ✓ ສາມາດປ່ຽນແປງໄດ້ ແຕ່ຈະຫາຍໄປເມື່ອລືບລ້າງ



Docker Image



Docker Container



Docker Container



ການປຽບທຽບກັບການຂຽນໂປຣແກຣມ: Docker Image ຄື "Class" ແລະ Docker Container ຄື "Object" ທີ່ຖືກສ້າງຂຶ້ນມາຈາກ Class ນັ້ນ

Dockerfile Example: ແອັບພລິເຄຊັນ Node.js

ການສ້າງ Dockerfile ພື້ນຖານສໍາລັບແອັບພລິເຄຊັນ Node.js



Dockerfile

```
1 # Step 1: ເລືອກ Base Image
2 FROM node:18-alpine
3
4 # Step 2: ກໍານົດ Working Directory
5 WORKDIR /app
6
7 # Step 3: ສໍາເນົາ package.json
8 COPY package*.json ./
9
10 # Step 4: ຕິດຕັ້ງ Dependencies
11 RUN npm install
12
13 # Step 5: ສໍາເນົາ Source Code
14 COPY . .
15
16 # Step 6: ເປີດ Port
17 EXPOSE 3000
18
19 # Step 7: ຄໍາສັ່ງທີ່ລັນເມື່ອເລີ່ມເຮັດວຽກ
20 CMD [ "node", "server.js" ]
```



ຂັ້ນຕອນທີ່ສໍາຄັນ

- 1 **Base Image:** ເລືອກ Node.js v18 ພ້ອມ Alpine Linux ເພື່ອຂະໜາດນ້ອຍ
- 2 **Working Directory:** ກໍານົດໄຟລເດີໃນ Container ເປັນ /app
- 3 **Copy package.json:** ສໍາເນົາໄຟລ໌ກ່ອນ ເພື່ອໃຊ້ Caching
- 4 **Install Dependencies:** ຕິດຕັ້ງ Libraries ທີ່ຈໍາເປັນ
- 5 **Copy Source Code:** ສໍາເນົາໂຄດທັງໝົດ



ຄໍາສັ່ງທີ່ກ່ຽວຂ້ອງ



```
docker build -t my-node-app .
```



```
docker run -p 8080:3000 my-node-app
```

Cloud Computing: ຮູບແບບການບໍລິການ

IaaS, PaaS ແລະ SaaS - ການປຽບທຽບກັບການເຮັດ Pizza



IaaS

Infrastructure as a Service

ຜູ້ໃຫ້ບໍລິການຈັດການ ໂຄງສ້າງພື້ນຖານ
(Server, Storage, Networking)

ໄດ້ ເຮືອນຄົວ ແລະ ເຕົາອົບ, ແຕ່
ຕ້ອງເອົາວັດຖຸດິບມາເອງ

Amazon EC2

Google Compute Engine

Azure VMs



PaaS

Platform as a Service

ຜູ້ໃຫ້ບໍລິການຈັດການ ແພລດຟອມ
(OS, Runtime, Middleware)

ສັ່ງ Pizza delivery, ພຽງແຕ່
ເລືອກໜ້າທີ່ຕ້ອງການ

Heroku

Google App Engine

AWS Elastic Beanstalk



SaaS

Software as a Service

ຊອບແວສໍາເລັດຮູບທີ່ ພ້ອມໃຫ້ໃຊ້ງານ
ຜ່ານອິນເຕີເນັດ

ໄປກິນ Pizza ຢູ່ຮ້ານ, ບໍ່ຕ້ອງເຮັດຫຍັງ
ເລີຍ

Gmail

Google Docs

Microsoft 365

Salesforce

← ລະດັບຂອງການຈັດການ



On-Premises
ຈັດການທຸກຢ່າງ



IaaS
ຈັດການ App ແລະ Data



PaaS
ຈັດການ App ແລະ Data



SaaS
ບໍ່ຕ້ອງຈັດການຫຍັງ

Cloud ແລະ DevOps: ຄູ່ຫູທີ່ເກີດມາເພື່ອກັນແລະກັນ

Cloud Computing ເປັນ "ຜູ້ເປີດທາງ" ທີ່ເຮັດໃຫ້ແນວທາງປະຕິບັດຂອງ DevOps ເປັນໄປໄດ້ຈິງ



Cloud Computing

ມອບ "ພະລັງ" ແລະ "ຄວາມໄວ" ໃຫ້
ກັບທີມພັດທະນາ



On-demand Infrastructure

ສ້າງ Server ໄດ້ທັນທີ ໂດຍການຄລິກບໍ່ກີ້ຄັ້ງ, ຊ່ວຍໃຫ້ CI/CD ສ້າງສະພາບ
ແວດລ້ອມແບບ "ໃຊ້ແລ້ວຖິ້ມ" ໄດ້



Infrastructure as Code (IaC)

ຂຽນໂຄດເພື່ອຈັດການ Server, Database, ແລະ Network ໂດຍອັດຕະໂນມັດ
, ນຳໃຊ້ແນວທາງປະຕິບັດ DevOps ມາໃຊ້ກັບ Infrastructure



Auto-scaling

ຕັ້ງຄ່າໃຫ້ລະບົບ "ເພີ່ມ" ຫຼື "ຫຼຸດ" ຈຳນວນ Server ໂດຍອັດຕະໂນມັດ
ຕາມປະລິມານການໃຊ້ງານ



PaaS ເຮັດໃຫ້ DevOps ງ່າຍຂຶ້ນ

ເຊັ່ນ Heroku ໄດ້ເຊື່ອງຄວາມຊັບຊ້ອນຂອງການຈັດການ Server ໄວ້ທັງໝົດ,
Developer ສາມາດ deploy ໄດ້ງ່າຍໆ

"Cloud Computing ໄດ້ປ່ຽນແປງພື້ນຖານໂຄງລ່າງຂອງການພັດທະນາຊອບແວ ແລະ ເປັນພື້ນຖານທີ່ຂາດບໍ່ໄດ້ທີ່ເຮັດໃຫ້ວັດທະນະທຳ ແລະ
ແນວທາງປະຕິບັດຂອງ DevOps ສາມາດເກີດຂຶ້ນໄດ້ຈິງ"

ສະຫຼຸບ: ຈຸດສໍາຄັນຂອງ DevOps

ສິ່ງທີ່ຕ້ອງຈົ່ງຈໍາໃນການປະຕິບັດ DevOps



ທໍາລາຍກໍາແພງ

- ✓ DevOps ບໍ່ແມ່ນຕໍາແໜ່ງ ແຕ່ **ວັດທະນະທໍາ**
- ✓ ສົ່ງເສີມການຮ່ວມມື ແລະ ຮັບຜິດຊອບຮ່ວມກັນ



ວັດທະນະທໍາ CALMS

- ✓ **Culture** - ການຮ່ວມມື ແລະ ຮັບຜິດຊອບຮ່ວມກັນ
- ✓ **Automation** - ຫຼຸດຜ່ອນຄວາມຜິດພາດ ແລະ ເພີ່ມຄວາມໄວ



CI/CD

- ✓ **CI** - ລວມໂຄດຢູ່ສະເໝີ, ຫຼຸດຜ່ອນຄວາມສ່ຽງ
- ✓ **CD** - ສົ່ງມອບແບບຕໍ່ເນື່ອງ, ຫຼຸດໄລຍະເວລາ



Docker & Cloud

- ✓ **Container** - ແກ້ໄຂບັນຫາ "It works on my machine"
- ✓ **Cloud** - ເປັນຜູ້ເປີດທາງ ທີ່ເຮັດໃຫ້ DevOps ເປັນໄປໄດ້ຈິງ

"DevOps ແມ່ນການປ່ຽນແປງວັດທະນະທໍາຂອງອົງກອນ ທີ່ຊ່ວຍໃຫ້ສາມາດ ສົ່ງມອບຊອບແວທີ່ມີຄຸນຄ່າໃຫ້ລູກຄ້າໄດ້ຢ່າງຕໍ່ເນື່ອງ"